PII Detection and Masking in Large Language Models

Table of Contents

Group Members:

Shrawan Kumar, Nikhil Reddy Kandadi, Venkata Vyshnavi Nemani, Ashwini Fernandes, Zeshan Choudhry

SmartPHIMasker

SmartPHIMasker is a notebook-based prototype for detecting and masking Protected Health Information (PHI) before healthcare text or structured patient data is sent to a Large Language Model. The project uses Microsoft Presidio, spaCy, and custom healthcare-focused recognizers to reduce PHI leakage while preserving enough context for downstream clinical or administrative tasks.

What this project does

The notebook builds a PHI guardrail pipeline for LLM workflows in healthcare. It:

- detects PHI in free text and structured records
- masks detected values with readable placeholders such as `{Name_xxx}`, `{DATE_xxx}`, `{PHONE_NUMBER}`, and `{INSURANCE_ID}`
- supports recursive masking for dictionaries and lists
- keeps a mapping so masked content can be restored if needed
- evaluates whether masked prompts prevent PHI leakage in LLM outputs
- compares behavior across OpenAI and local Hugging Face based model demos

Main idea

Instead of sending raw patient records to an LLM, the notebook first passes the content through `SmartPHIMasker`. Sensitive values are replaced with semantic placeholders. The masked content is then sent to the model, allowing report generation or workflow automation with much lower exposure risk.

Core components

1. Dependency setup

The notebook installs and uses:

- `presidio-analyzer`
- `presidio-anonymizer`
- `spacy`
- `en_core_web_lg`
- `word2number`
- `dateparser`
- `openai`
- `transformers`
- related utilities such as `phonenumbers`, `pandas`, and `torch`

2. Custom PHI recognizers

A custom recognizer module extends Presidio for healthcare-specific fields. Based on the notebook, the system is designed to handle items such as:

- names
- addresses and geographic details
- date of birth and other dates
- age
- phone numbers
- email addresses
- Social Security Numbers
- medical record numbers
- insurance IDs / health plan beneficiary numbers
- account numbers
- URLs
- IP addresses
- NPI numbers
- credit card data

3. Stop-word and allow-list logic

The masker uses a large stop-word / allow-list set so common medical terms and non-sensitive words are preserved. The notebook also manually adds a few medical terms such as `Hypertension`, `Diabetes`, `Metformin`, and `Lisinopril` to reduce false positives.

4. `SmartPHIMasker` engine

The main class in the notebook is `SmartPHIMasker`. Important capabilities include:

- Presidio initialization with spaCy `en_core_web_lg`
- custom recognizer registration
- masking of free text with `smart_mask()`
- recursive masking of dicts and lists with `smart_mask_struct()`
- reversal of masking with `smart_unmask()`
- overlap removal for competing recognizer spans
- consistent placeholder generation for repeated entities

5. Placeholder strategy

The notebook uses readable placeholders instead of fully opaque tokens. Examples include:

- `{Name_<hash>}`
- `{DATE_OF_BIRTH}`
- `{DATE_<hash>}`
- `{PHONE_NUMBER}`
- `{SSN}`
- `{MEDICAL_RECORD_NUMBER}`
- `{INSURANCE_ID}`
- `{ACCOUNT_NUMBER}`

- `{URL}`
- `{IP_ADDRESS}`
- `{NPI_NUMBER}`
- `{CREDIT_CARD}`
- `{Address_<hash>}`

This makes prompts more usable for LLMs while still hiding the original sensitive value.

6. Evaluation utilities

The notebook defines two evaluation helpers:

- `evaluate_pipeline_brief(...)`
- `evaluate_pipeline_detailed(...)`

These functions compare original PHI fields against model outputs to estimate:

- masking success rate
- PHI leakage
- number of masked values
- approximate detection quality

Notebook workflow

The notebook is organized as a step-by-step prototype:

1. Install dependencies
2. Import libraries
3. Define custom PHI recognizers
4. Load stop words and helpers
5. Build the `SmartPHIMasker` engine
6. Initialize the masking system
7. Connect to OpenAI for LLM inference
8. Run evaluation and test cases
9. Try local LLM experiments with Hugging Face models
10. Run a final cleaned OpenAI demo
11. Optionally expose a Gradio UI

Included test scenarios

The notebook contains multiple healthcare-oriented test cases demonstrating both unmasked and masked prompting. These include workflows such as:

- clinical report generation
- discharge summary generation
- lab results review and clinical insight

- billing and payment processing
- travel history and identity verification
- longitudinal EHR review and summary generation
- patient portal access and contact data handling
- intake and identity verification
- edge cases for short, multi-part, or non-Western names
- non-PII preservation checks such as blood group remaining unchanged

Example usage

A simplified usage pattern based on the notebook is:

```
masker = SmartPHIMasker(
    allow_list=["Hypertension", "Diabetes", "Metformin"],
    mask_dates=True,
    mask_geographic=True,
    mask_ages=True,
    mask_medical_ids=True
)

record = {
    "Patient Name": "John Doe",
    "DOB": "05/15/1985",
    "MRN": "12345678",
    "Insurance ID": "BCBS987654321",
    "Diagnosis": "Hypertension"
}

masked_record, mapping = masker.smart_mask_struct(record)
print(masked_record)
print(mapping)
```

Expected outputs

After masking, structured or textual healthcare data should preserve task-relevant context while removing direct identifiers. For example:

- names become semantic placeholders
- dates become masked date tokens
- MRNs and insurance IDs become standard placeholders
- diagnoses and non-sensitive clinical information remain readable

Strengths of the notebook

- combines Presidio with custom healthcare rules
- handles both text and structured JSON-like data
- keeps prompts readable for LLMs
- includes evaluation logic instead of masking only

- explores both hosted and local model workflows
- considers edge cases like unusual personal names and non-PII preservation

Current limitations

Based on the notebook content, there are a few practical limitations:

- the project is implemented as a single notebook rather than a packaged module
- some recognizers appear to be rule-based and may need further tuning for recall and precision
- location and organization detection from spaCy are intentionally not fully handled as PHI in the current implementation notes
- biometric identifiers and face or image-based identifiers are explicitly not handled
- several sections are demo-oriented and may need cleanup before production use

Important security note

The notebook currently appears to contain hardcoded credentials or tokens for external services. That is unsafe for sharing or version control.

Before publishing or deploying this project, replace all embedded secrets with environment variables. For example:

```
import os
from openai import OpenAI

client = OpenAI(api_key=os.getenv("OPENAI_API_KEY"))
```

Similarly, store Hugging Face tokens in environment variables or a secure secrets manager.

Recommended cleanup before productionizing

- remove all hardcoded API keys and tokens
- move the recognizers and masking engine into reusable `.py` modules
- add a `requirements.txt`
- add unit tests for each PHI type and edge case
- separate demo code from core library code
- add benchmark metrics such as precision, recall, and false positive rate
- document supported entity types more formally
- add secure configuration handling and logging controls

Suggested project structure

A cleaner structure for this project could be:

```
SmartPHIMasker/
├── notebooks/
│   └── latest_demo.ipynb
├── smart_phi_masker/
```

```
│   ├── __init__.py
│   ├── recognizers.py
│   ├── masker.py
│   ├── evaluation.py
│   └── utils.py
├── tests/
├── requirements.txt
└── README.md
```

How to run

- Step 1: Install Dependencies
- Step 2: Import Libraries
- Step 3: Protected health Information Recognizers (Custom ones not in Spacy are in bold):
- Step 4: STOP WORDS and Helper(s) module
- Step 5: Step 5: SmartPHIMasker Engine: Core system for detecting and masking PHI.
- Step 6: Initialize Masking System
- Step 7: LLM Integration using OpenAI - Creates openAI client
- From step 8 which is Evaluate and Masking, we ran 12 test cases. So, in each test cases, we unmask the data and it is first sent to the LLM, followed by masking of the data, demonstrating how our smart masker prevents PII leakage.
- Step 8.5: Testing the baseline model with FLAN-T5
- Step 9: Local LLM Integration (Hugging Face Models)
- Step 10: OpenAI LLM Integration (Primary System)
- Step 11: Optional UI Demo (Gradio)- This take lots of time to load, so it is commented, but can be uncommnted to see(if needed)

Future improvements

- add support for more HIPAA identifiers
- improve multilingual and non-Western name handling
- add a configurable policy layer for different masking strategies
- integrate with hospital intake, billing, or EHR middleware
- build a lightweight API or web app for real-time PHI masking

Summary

This notebook demonstrates a practical prototype for PHI-aware LLM safety in healthcare. Its main contribution is a hybrid masking pipeline that combines custom recognizers,

Presidio-based entity analysis, readable placeholders, and leakage-focused evaluation across realistic clinical and administrative scenarios.